

Systèmes Informatiques

7. Systèmes d'exploitation – entrées/sorties

Guillaume Pierre

ISTIC, L3 MIAGE



- 1 Les périphériques d'entrée-sortie
- 2 Ordonnancement des entrées/sorties
- 3 Les deadlocks

Les périphériques d'entrée-sortie

Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
56K modem	7 KB/sec
Scanner	400 KB/sec
Digital camcorder	4 MB/sec
52x CD-ROM	8 MB/sec
FireWire (IEEE 1394)	50 MB/sec
USB 2.0	60 MB/sec
XGA Monitor	60 MB/sec
SONET OC-12 network	78 MB/sec
Gigabit Ethernet	125 MB/sec
Serial ATA disk	200 MB/sec
SCSI Ultrawide 4 disk	320 MB/sec
PCI bus	528 MB/sec

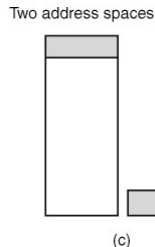
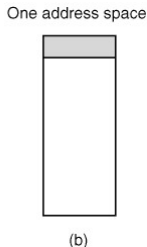
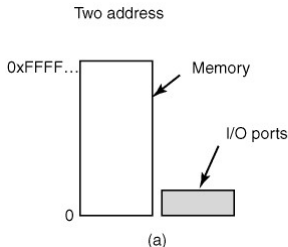
Deux sortes de périphériques

- Les **block devices** manipulent des blocs de taille fixe
 - Exemple: disques (1 secteur = 512 octets)
- Les **character devices** manipulent des données de n'importe quelle taille
 - Claviers
 - Souris
 - Imprimantes
 - Cartes réseau
 - Etc.

- Tous les périphériques sont différents les uns des autres
 - Parties mécaniques etc.
- Chaque périphérique contient un **contrôleur** en charge de communiquer avec le reste de la machine
 - Envoie les commandes au bras du disque
 - Convertit les bits en blocs
 - Etc.
- Les contrôleurs peuvent être très sophistiqués
 - Buffering
 - Exécution spéculative
 - etc.

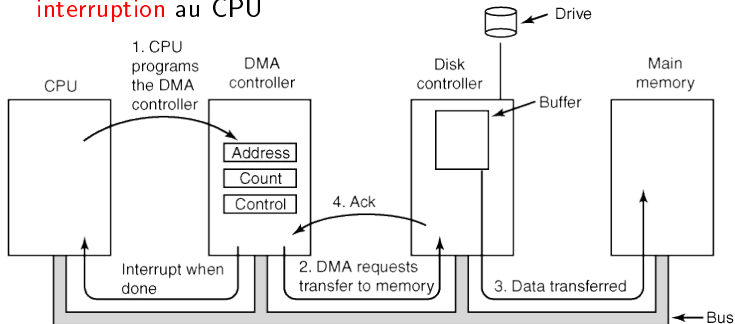
Les périphériques "Memory-mapped"

- Chaque contrôleur a des registres mémoire internes
 - Le CPU lit/écrit dans ces registres (grâce au bus) pour communiquer avec le périphérique
- Une abstraction utile: prétendons que **ces registres font partie de l'espace d'adressage normal de la machine**
 - Il faut réserver un intervalle d'adresses mémoire (hors de la mémoire physique de la machine)
 - Pour communiquer avec le périphérique, le CPU doit simplement lire/écrire dans ces adresses mémoire
 - Le bus s'occupe du reste



Direct Memory Access (DMA)

- Les registres du contrôleur font partie de l'espace d'adressage uniforme de la machine
 - Mais ce n'est qu'une abstraction! Il faut quand même transférer les données à travers le bus
- DMA = permettre aux **périphériques de parler directement avec la mémoire**
 - Sans devoir transiter par le CPU
 - Quand le transfert est terminé: le contrôleur DMA envoie une **interruption** au CPU



Objectifs du système d'entrées/sortie

1 Indépendance du matériel

- Lire des données d'un CD-ROM, d'un disque ou d'une clé USB doit fonctionner de la même façon!

2 Nommage uniforme

- Tous les périphériques doivent avoir des nom uniformes
- Par exemple: /media/cdrom/repertoire/fichier.pdf

3 Gestion d'erreurs

- Gérer les erreurs aussi près du hardware que possible
- Ne pas déranger les couches hautes sauf si c'est indispensable!

4 Entrées/sorties synchrones ou asynchrones

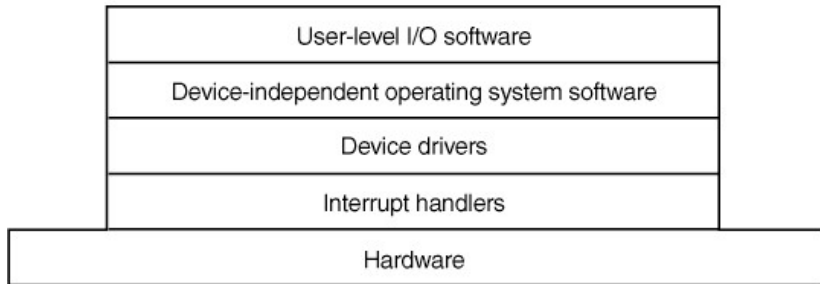
- Toutes les entrées/sorties sont asynchrones: le CPU peut faire autre chose en attendant
- Mais les programmes utilisateurs préfèrent souvent l'illusion que les entrées/sorties sont synchrones

5 Buffering

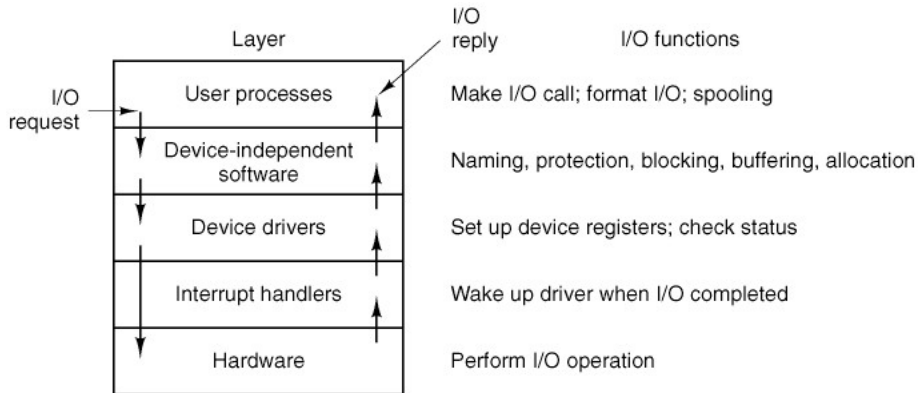
- Souvent on ne peut pas choisir le moment où des données arrivent d'un périphérique
- Exemple: carte réseau
- Il faut stocker les données en attendant qu'elles soient consommées

Organisation du système d'entrées/sorties

- Le système d'entrées/sorties est généralement organisé en quatre couches



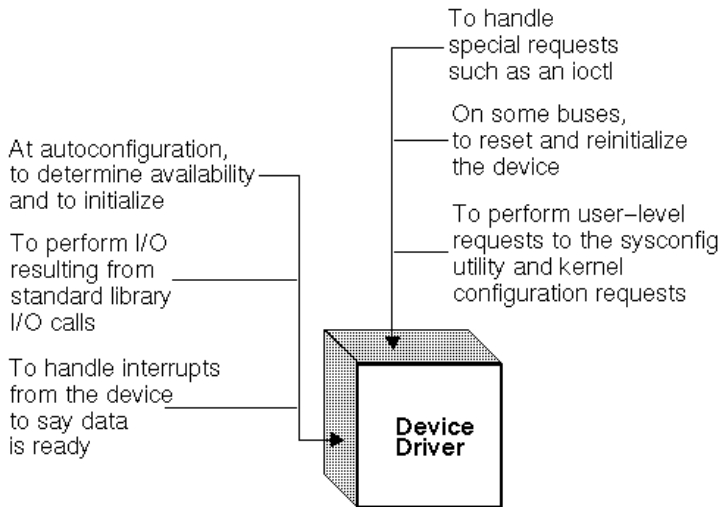
Fonctionnement du système d'entrées/sorties



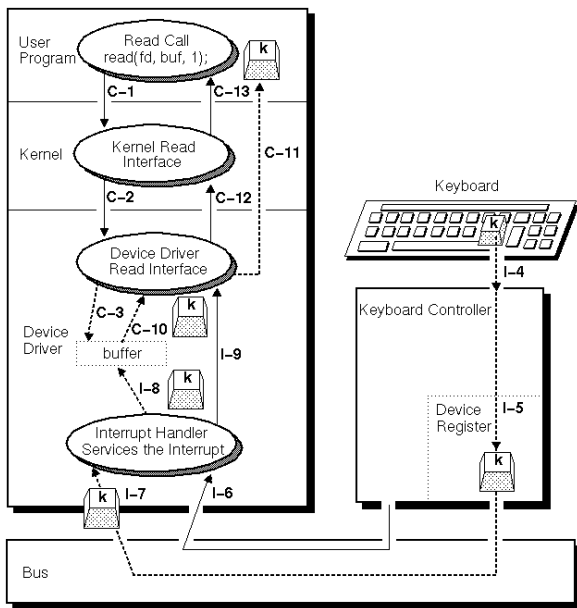
- Programmer à base d'interruptions est **difficile**
 - Votre programme peut être interrompu à n'importe quel moment
 - Il faut maintenir le contexte de chaque opération en cours dans le système
- Plus facile pour les utilisateurs: **rendre les entrées-sorties synchrones** (artificiellement)
 - Quand un programme demande une entrée/sortie: bloquer le driver avec un sémaphore
 - Quand on reçoit l'interruption: débloquer le driver en débloquant le sémaphore
 - C'est plus facile à implémenter si chaque driver de périphérique est implémenté sous forme d'un processus séparé

- Les périphériques sont **très différents les uns des autres**
 - Un driver de souris reçoit des informations de mouvement, de distance, de direction, de clics etc.
 - Un driver de disque manipule des secteurs, des pistes, des cylindres, etc. cylinders, heads, arm motion, motor drives, etc.
 - Même deux périphériques similaires (p.ex. deux cartes réseaux) peuvent fonctionner très différemment et doivent recevoir des commandes différentes
- Comment le système sait-il comment utiliser chaque périphérique?
 - Chaque fabricant écrit un **device driver** qui sait comment utiliser le périphérique
- Chaque driver implémente l'**API standard** le mieux possible
 - Grâce à la connaissance du fonctionnement interne du périphérique...

Interface d'un device driver



Exemple: lire un caractère du clavier



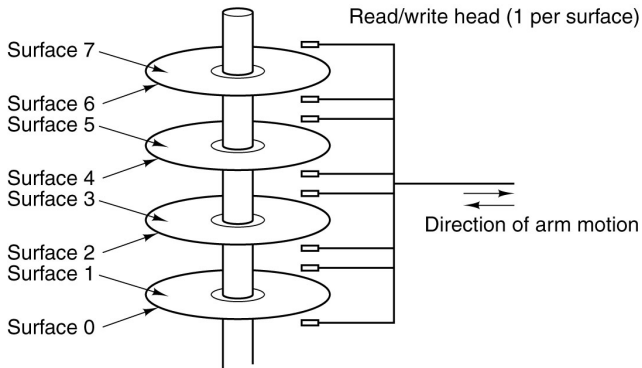
Nommage des périphériques

- Il faut donner un **nom** à chaque périphérique
- Dans Unix: chaque périphérique est représenté par un **fichier spécial**:
 - `/dev/hda`, `/dev/hdb` représentent des disques IDE/SATA
 - `/dev/sda`, `/dev/sdb` représentent des disques SCSI/USB (p.ex, clés USB)
 - etc.
- On peut créer d'autres **fichiers spéciaux**
 - `/dev/null`, `/dev/random`, etc.
- Chaque périphérique est décrit par deux nombres:
 - **Major device number**: définit le type de périphérique (cela permet de choisir le bon driver)
 - **Minor device number**: indique un périphérique particulier
- Donner des noms de fichiers aux périphériques permet également de leur donner des **droits d'accès**

- 1 Les périphériques d'entrée-sortie
- 2 Ordonnancement des entrées/sorties
- 3 Les deadlocks

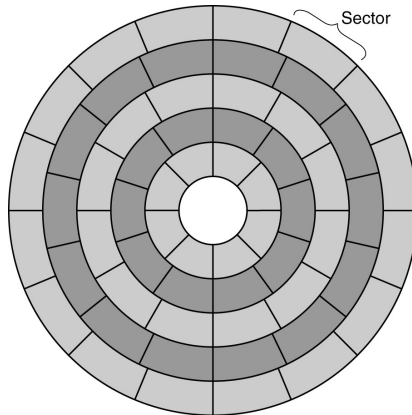
Les disques durs

- Un disque dur est composé de:
 - 6-12 **plateaux** (qui tournent à 5400-10000 rotations/min)
 - 1 **bras** et **une tête** pour lire/écrire les données



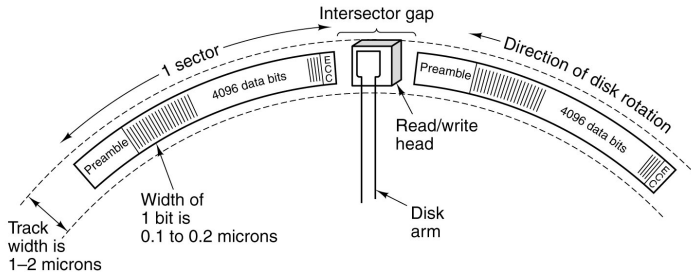
Les disques durs

- Chaque plateau contient **10-30 zones**
- Chaque zone contient **plusieurs secteurs** (les zones extérieures ont plus de secteurs que les zones intérieures)



■ Chaque secteur contient:

- Un **préambule** (pour permettre à la tête de reconnaître le début du secteur)
- **512 octets de données**
- Un **code de correction d'erreur (ECC)**



■ La durée pour accéder à un secteur:

- Attendre que le disque tourne jusqu'au bon angle
 - A 7200 rpm: 1 tour = 0.14 ms
- Attendre que le bras se déplace jusqu'à la bonne piste: **beaucoup plus long: 5–20 ms!**
 - C'est difficile d'aller très vite parce que le bras ne sait jamais où il devra aller ensuite

■ Comment réduire les délais:

- 1 Faire tourner le disque plus vite (pas beaucoup de gains à en attendre)
- 2 Placer les données côte à côte (défragmentation)
- 3 **Ré-ordonnancer les accès quand on reçoit plusieurs requêtes pour des pistes différentes** (par ex: plusieurs utilisateurs accèdent à des fichiers différents en même temps)

First-Come-First-Served (FCFS)

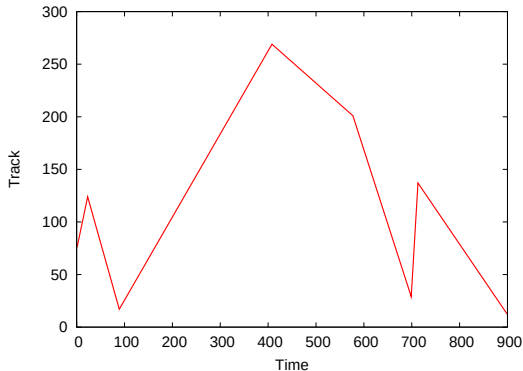
- Idée: **traiter les requêtes dans l'ordre où elles arrivent**



Simple

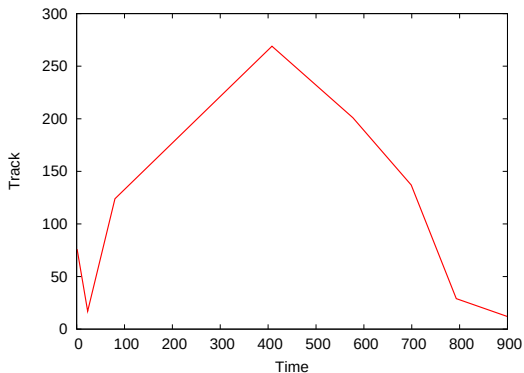


Beaucoup de déplacements du bras! Mauvaise performance



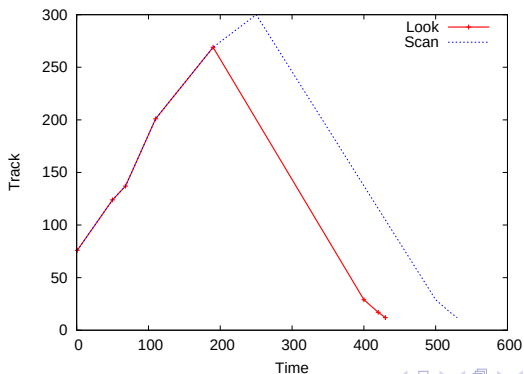
Shortest-Seek-Time-First (SSTF)

- Idée: **traiter la requête en attente la plus proche de la position actuelle**
 - 😊 Moins de déplacements du bras $\Rightarrow$ plus rapide
 - 😞 Sous forte charge: **certaines requêtes peuvent ne jamais être traitées**



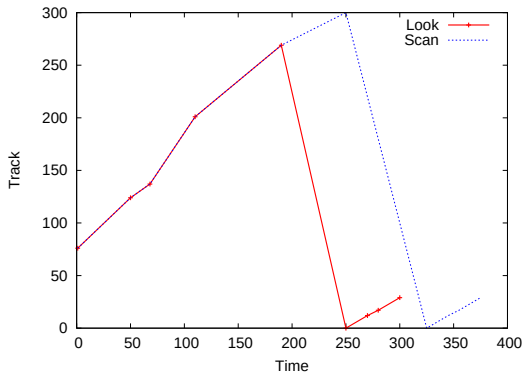
Scan et Look

- Scan: déplacer le bras le long de toutes les pistes, traiter les requêtes pour les pistes devant lesquelles on passe
 - 😊 Toutes les requêtes sont traitées
 - 😊 Bonne performance sous forte charge
 - 😞 Temps d'accès parfois importants!
- Look: changer de direction s'il n'y a plus de requête à traiter dans la direction actuelle



Scan et Look circulaires

- Certains disques ont une fonction très efficace “ramener la tête à la piste 0”
- Idée: **scan/look dans une seule direction**
 - Plus équitable entre les différentes requêtes



- 1 Les périphériques d'entrée-sortie
- 2 Ordonnancement des entrées/sorties
- 3 Les deadlocks**

Les deadlocks (inter-blocages)



Les deadlocks (inter-blocages)



Deadlocks

- Certains périphériques peuvent réaliser **plusieurs actions à la fois**
 - Par ex: un disque dur gère des accès simultanés
- Mais beaucoup de ressources ne peuvent réaliser **qu'une seule opération à la fois!**
 - Imprimante
 - Clavier
 - Certaines données dans les tables du noyau
 - etc.
- Le système permet de **verrouiller** l'accès à une ressource, mais cela peut créer des **deadlocks**
- Exemple: deux programmes veulent scanner une image et l'écrire sur un CD-ROM



Deadlocks

- Certains périphériques peuvent réaliser **plusieurs actions à la fois**
 - Par ex: un disque dur gère des accès simultanés
- Mais beaucoup de ressources ne peuvent réaliser **qu'une seule opération à la fois!**
 - Imprimante
 - Clavier
 - Certaines données dans les tables du noyau
 - etc.
- Le système permet de **verrouiller** l'accès à une ressource, mais cela peut créer des **deadlocks**
- Exemple: deux programmes veulent scanner une image et l'écrire sur un CD-ROM

Programme 1	Programme 2
Verrouiller le scanner —	— Verrouiller le CD-ROM
Impossible de verrouiller le CD-ROM!	Impossible de verrouiller le scanner!

Quand est-ce qu'un deadlock peut se produire?

Pour avoir un deadlock, il faut **quatre conditions**:

1 Exclusion mutuelle

- Une ressource ne peut être verouillée que par un seul processus à la fois

2 Hold and wait

- Un processus qui a verouillé une ressource peut demander à verrouiller d'autres ressources

3 Pas de pré-emption

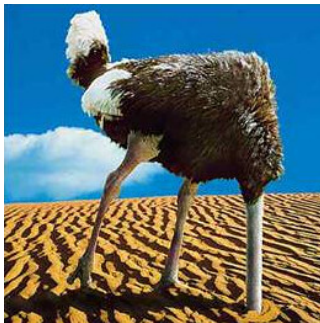
- Une fois qu'un processus a verrouillé une ressource, le système ne peut pas la reprendre de force

4 Attentes circulaires

- Il faut un cycle de processus qui s'attendent les uns les autres

Comment résoudre les deadlocks

- Solution #1: Ignorer le problème complètement
- Ce n'est pas forcément une mauvaise solution!
 - Est-ce que les deadlocks arrivent souvent?
 - Est-ce qu'ils sont très gênants?



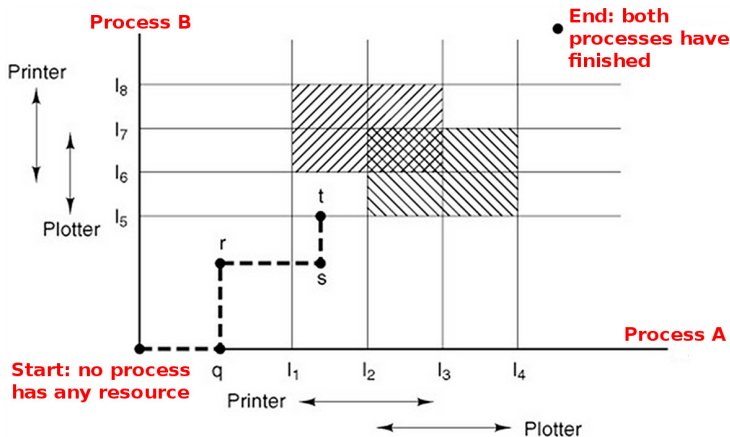
- Solution #2: Détecter et corriger
- Explorer périodiquement le graphe des processus en attente de ressources
 - Si on détecte un cycle: choisir l'un des processus du cycle, le tuer
- Encore plus simple (et plus brutal): si un processus est bloqué depuis 1 heure, alors on le tue

Pas très élégant...

- Solution #3: **Empêcher les deadlocks de se produire**
- 1 **Spooling**: faire en sorte que les processus n'aient pas besoin de verrouiller les ressources
 - Au lieu de laisser les processus verrouiller l'imprimante, on leur demande d'inscrire leurs fichiers à imprimer dans un "spool"
 - Un seul processus peut verrouiller l'imprimante: le serveur d'impression
 - Mais cela ne s'applique pas à toutes les ressources. Et que faire si le spool est plein?
- 2 **Verrouiller toutes les ressources nécessaires atomiquement**
 - Pas facile à réaliser
- 3 **Ordonnancer les ressources** (p.ex en ordre alphabétique) et imposer que les processus ne verrouillent les ressources qu'en ordre croissant

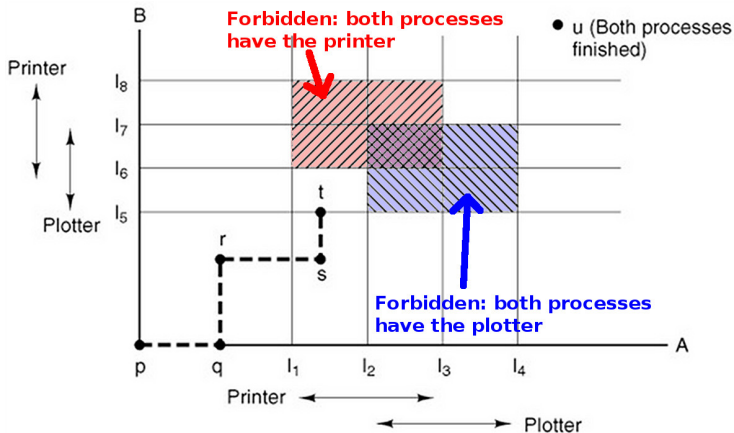
Comment résoudre les deadlocks

- Solution #4: **Eviter d'entrer en deadlock**: l'algorithme du banquier
 - Prévoir les **futures requêtes que d'autres processus pourraient faire**, et éviter d'entrer dans des états qui peuvent mener à des deadlocks



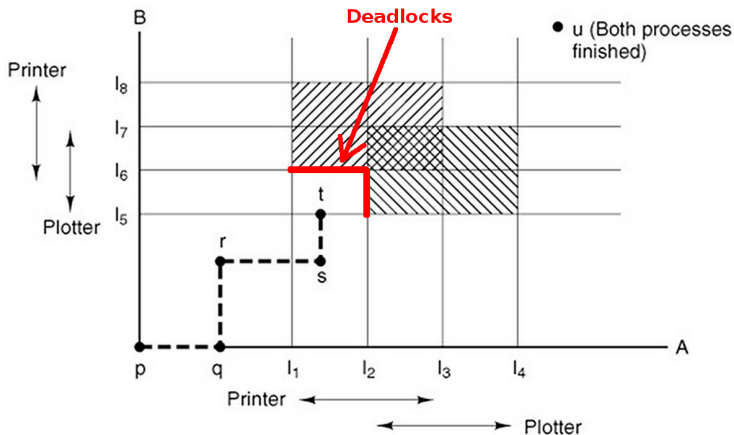
Comment résoudre les deadlocks

- Solution #4: **Eviter d'entrer en deadlock**: l'algorithme du banquier
 - Prévoir les **futures requêtes** que d'autres processus pourraient faire, et éviter d'entrer dans des états qui peuvent mener à des deadlocks



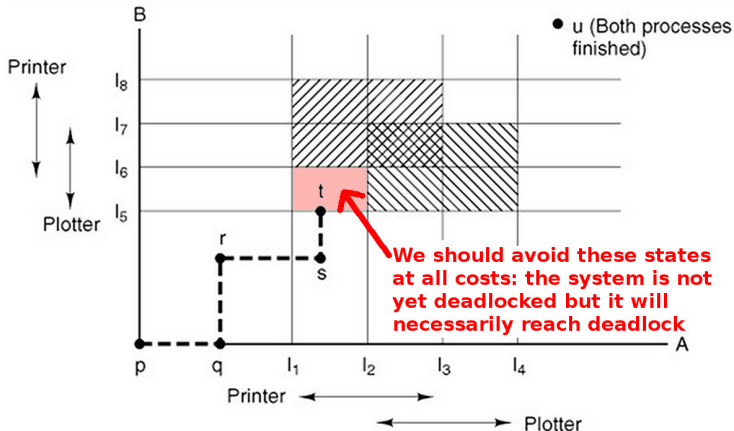
Comment résoudre les deadlocks

- Solution #4: **Eviter d'entrer en deadlock**: l'algorithme du banquier
 - Prévoir les **futures requêtes** que d'autres processus pourraient faire, et éviter d'entrer dans des états qui peuvent mener à des deadlocks



Comment résoudre les deadlocks

- Solution #4: **Eviter d'entrer en deadlock**: l'algorithme du banquier
 - Prévoir les **futures requêtes** que d'autres processus pourraient faire, et éviter d'entrer dans des états qui peuvent mener à des deadlocks



Cette technique suppose qu'on sait à l'avance quelles ressources peuvent être **potentiellement** verrouillées par quels processus